



FAKULTI SAINS KOMPUTER DAN MATEMATIK (FSKM)

SEMESTER II 2025/2026

CSE3433 SOFTWARE ARCHITECTURE (K2)

Individual Logbook

NOR AIN NATASYA BINTI KAMARULZAMAN

(S76002)

GROUP 8

Service-Oriented Smart Campus Support System

(Service-Oriented Architecture)

LECTURER:

Dr. MOHAMAD NOR BIN HASSAN

GROUP MEMBERS:

NAME	NO. MATRIC
NOR SAERAH BINTI ISMAIL	S75325
NOR AIN NATASYA BINTI KAMARULZAMAN	S76002
CHE KU NURULHUDA BINTI CHE KU SAIFUL MUZAHAR	S75871

Weekly Progress

PHASE 1 : PROJECT PROPOSAL (Week 3 - Week 5)

Week	Activities Undertaken	Individual Contribution	Challenges	Action Plan
3	Researched UI/UX patterns for a campus support system. Evaluated user interaction paths for an issue reporting system. Evaluated how well the front-end design aligned with Service Oriented Architecture principles.	Helps achieve system goals by focusing on user experience and the importance of responsive design. Propose a simple portal interface to keep complex backend services hidden from users.	Ensuring the right balance between attractive design and the limitations of the backend of a service-oriented architecture. Ensuring that the frontend does not violate the principles of loose coupling by connecting directly to the backend services.	Develop a front end that communicates exclusively through the API Gateway. Make sure to avoid any direct database connections or calls to service-specific APIs from the UI.
4	Created the first wireframe for the Campus Support System. Outline how users would navigate through submitting issues and track their status.	I made some basic wireframes for the main page of the Campus Support System featuring a department selector, a form for submitting issues and a page to confirm success.	Ensure that the wireframe shows the SOA service boundaries between IT and Facilities departments. Users need a clear visual separation between different service areas while avoiding any confusion with back-end complexity.	Department selection is color-coded, using blue for IT and green for Facilities, to help visually guide service routing while maintaining a simple interface.
5	Finalized frontend scope in proposal. Defined responsive design requirements. Specified UI technology stack (HTML/CSS/JSP).	Created functional requirements for UI/UX outlined the need for cross-browser compatibility, proposed JSP for server side rendering to work with the Java backend.	This proposal has limited space to explain all the front-end factors. It is necessary to provide reasons for choosing JSP rather than just using HTML and JS for SOA integration.	The main points of the front-end have been summarized, with a full wireframe included in the appendix. The choice of JSP was justified due to the team's expertise in Java and its seamless integration with RouterServlet.

PHASE 2: ARCHITECTURE DESIGN (Week 6 - Week 7)

Week	Activities Undertaken	Individual Contribution	Challenges	Action Plan
6	Examine how the front end interacts with API Gateway in SOA. Map UI actions to back end service calls. Analyze JSP form submission patterns for RESTful communication.	Documented frontend-to-gateway interaction flow. Specified HTTPS communication requirements. Designed form payload structure (department, description) to match the JSON format expected by the gateway.	Understand how stateless REST calls affect session management in JSP-based UIs. JSP traditionally uses server-side sessions, but SOA requires stateless requests.	Research stateless form submission patterns. Design each POST form to include all required data without relying on the server session state.
7	User interface mockups designed in line with architectural diagrams. Verify that UI flows match System Context Diagram interactions.	Created high-fidelity mockups for Campus Support Portal; ensured consistency with System Context Diagram (Students → System → Support Services). Designed success page to display routed department confirmation	Represents the concept of asynchronous notifications in a static mockup. The system sends a mock email notification via the console, but the UI needs to notify the user to check the NetBeans Output panel.	Italicized helper text has been added to the success page: "(Please check the NetBeans Output panel to see the mock email notification)". Screenshot evidence of the actual implementation matched.

PHASE 3: PROTOTYPE PROGRESS (Week 8 - Week 11)

Week	Activities Undertaken	Individual Contribution	Challenges	Action Plan
8	Developed the front end of the Campus Support Portal using JSP and HTML/CSS. Built the main interface with department dropdown form and issue description form.	Implement index.jsp with Department selector dropdown menu (<select> with "IT Issue" and "Facilities Issue" options), Issue description text input, Submit Report button with POST to RouterServlet. Responsive centered card layout styled with CSS.	The form action URL is required to correctly route to the RouterServlet in the SmartCampusSupportSystem context path. Relative vs. absolute path confusion causes 404 errors.	Action="RouterServlet" is used with the correct web.xml servlet mapping. The form submission path is tested multiple times in different browsers.
9	The front-end form is connected to the back-end RouterServlet. The validated form data (description parameters) reaches the gateway correctly. The department selection routing is tested.	Form field configured to send description parameter via POST method. Validated dropdown values are mapped to correct routing logic in RouterServlet (IT vs. Facilities department detection).	During prototype development, the front-end JSP form sent data as x-www-form-urlencoded while API testing in Postman used JSON format. This required the back-end to be flexible in handling different content types, even though the main production path used form-encoded data from the web portal.	Coordinated with backend developers. RouterServlet uses request.getParameter("description") for form data instead of JSON parsing. Ensures form enctype type matches backend expectations.
10	Refined UI based on test feedback. Improved success page layout and error handling display. Added "Back to Home" navigation.	A styled success page with a green confirmation message indicating the department being redirected to (IT Department or Facilities Department). A purple "Back to Home" hyperlink has	The success page is needed to display the department name that is dynamically routed from the servlet response. Static HTML cannot show "IT Department" vs. "Facilities Department" without	JSP expression language or servlet delivery with request attribute was used. RouterServlet was delivered to a successful JSP with department name as request attribute.

		been added for navigation flow. An error page template has been created for future 400/503 handling.	server-side rendering.	
11	Front-end screenshots captured for demonstration proof. UI verified during all test scenarios. System behavior documented for reporting.	Taking screenshots for System with Facility Issues selected and "Whiteboard broken" entered, System with IT Issues selected and "Projector broken" entered, Success Page showing "redirected to IT Department", Success Page showing "redirected to Facilities Department".	Timeline screenshot to capture the exact state of the form before submission. Browser autofill sometimes changes dropdown values unexpectedly.	Browser autofill for forms is disabled. Screenshot is captured immediately after setting the value and before submission. Snipping Tool with delay feature is used for post-submission success page.

PHASE 4: PROJECT REPORT, LOGBOOK & FINAL DEMO (Week 12 - Week 13)

Week	Activities Undertaken	Individual Contribution	Challenges	Action Plan
12	Document the front-end implementation in a project report. Explain UI design decisions, JSP technology choices, and gateway integration patterns.	Write Technology Stack (front end): HTML, CSS, JSP. Write the Prototype Implementation section which includes Campus Support System UI components, form structure and success page rendering. Include front end screenshots from Proof of Demonstration.	Explain why JSP was chosen over modern JavaScript frameworks (React/Vue) in the context of an academic report. It is necessary to justify the technology choice without appearing outdated.	The JSP choice is framed as a deliberate SOA alignment: server-side rendering keeps the front end thin, ensuring all business logic remains in the service, and maintaining loose coupling between presentation and back end.
13	Provided presentation slides showing UI flow. Live demo segment training. Created service startup sequence for demo reliability.	A slide deck created on the front-end architecture; ready for a live system demo during a presentation. The designed demo flow: show form → submit IT issue → show success → show database evidence → show NetBeans console notifications.	Concerns about live demo failure due to Tomcat service startup issues or port conflicts. JSP pages require the Apache Tomcat server to be running.	Provide a backup of the screen recording for the full demo. Initialization checklist created: Start MySQL, Start Tomcat, Verify localhost:8080/SmartCampusSupportSystem, Clear browser cache, Test a submission before the demo.

GENAI USAGE RECORD

Week	Date	Tool Used	Purpose	Prompt Used	Output Summary	How It was Used
3	29/03/2026	Gemini	Project Understanding	Explain SOA in a simple way.	SOA concept explained clearly including services, loose coupling, and standardized contracts.	Used during initial project planning and idea exploration.
3	30/03/2026	Gemini	Concept Exploration	Give me some examples of SOA systems.	Examples of SOA systems and campus support platforms provided.	Used to compare and evaluate possible project ideas.
4	07/04/2026	Gemini	System Analysis	What features are most likely needed in SOA systems?	Explained system services, user roles, and workflow requirements for ticketing systems.	Used to define project scope and system functionalities .
5	14/05/2026	Gemini	Requirements Analysis	I need some suggestions for non-functional requirements for SOA.	Non-functional requirements such as scalability, performance, security, and availability are identified.	Used in software architecture planning.
6	20/04/2026	Gemini	Architecture Design	How to split 4 services for SOA?	Explained RouterServlet, ITDepartmentServlet, FacilityDepartmentServlet, NotificationServlet.	Used in architecture design
6	20/04/2026	Gemini	SOA Architecture Design	How to use REST API in SOA architecture?	Suggested system components and architecture	Used to justify SOA implementation.

					diagram structure.	
7	27/04/2026	Gemini	System Modeling	How to draw a SmartCamp us component diagram?	Suggested system component diagram.	Used in architecture design preparation.
8	12/05/2026	Gemini	Development planning	I need you to teach me basic Java Servlet coding structure.	Suggested project folder structure and coding organisation.	Used during development setup.
9	19/05/2026	Gemini	API design	API endpoints for servlet POST method	Proposed servlet endpoints and HTTP POST methods.	Used in backend development planning.
9	20/05/2026	Gemini	System improvement	How to improve servlet systems?	Suggested user interface and backend improvements.	Used to enhance system quality.
10	26/05/2026	Gemini	Testing guidance	How to test servlets with postman?	Provided testing guidelines for the Postman tool.	Used during integration testing.
11	03/06/2026	Gemini	Testing guidance	How to handle form validation in JSP without complex JS?	Provided basic HTML5 validation attributes and simple servlet-side null checks.	Used to secure the web portal input forms against empty submissions.
12	08/06/2026	Gemini	Bug fixing	CSS grid layout breaks with long text string in JSP card	Suggested word-wrap , break-word and max-width constraints for responsive containers.	Applied to fix layout bugs on the success.jsp page.
13	15/06/2026	Gemini	Report Formatting	How to structure appendix for	Recommended layout for system walkthrough	Used to organize the user interface

				software architecture project UI ?	screenshots with brief step-by-step captions.	demonstration section in the Final Report.
--	--	--	--	--	---	--